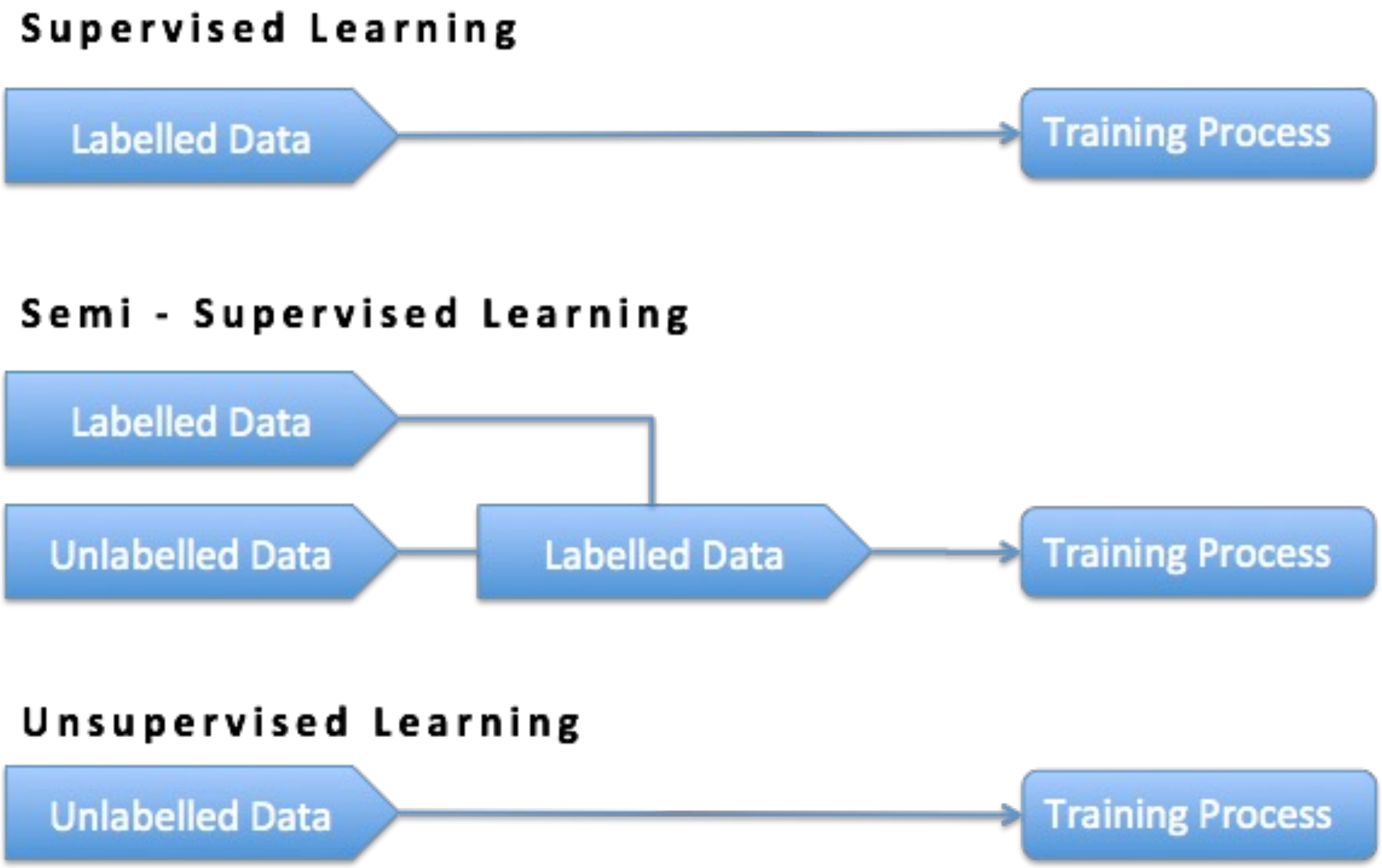
Computer Vision Lecture 4
Machine Learning Part 1

Jens Rittscher

Learning objectives

- ML overview
- Model validation
- Dimensionality reduction methods
- Classification using Gaussian mixtures
- Linear models
- Support vector machines
- Neural networks

Supervised vs. Unsupervised Learning



Supervised Learning

Goal: Learn a mapping from inputs to outputs

$$f : x \longrightarrow y$$

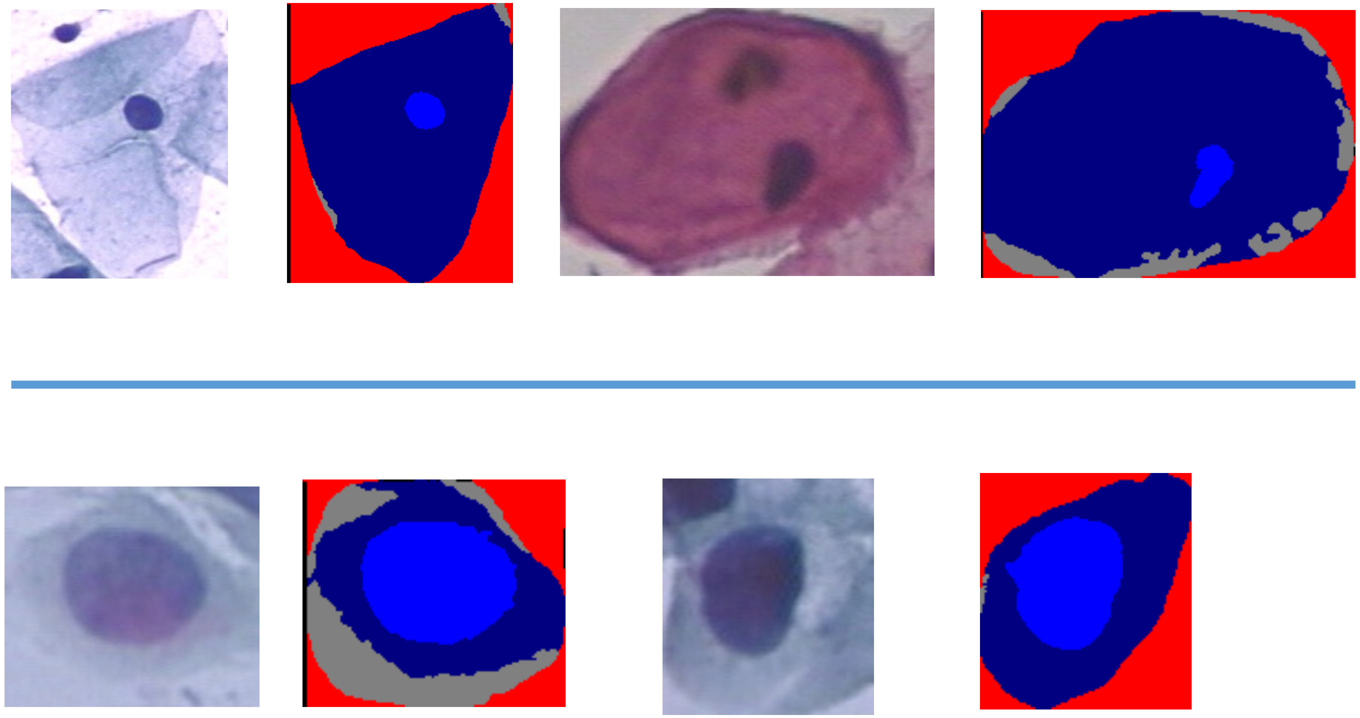
Given a training set , a of labelled set of input-output pairs:

$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{N}$$

Example: the pap smear dataset

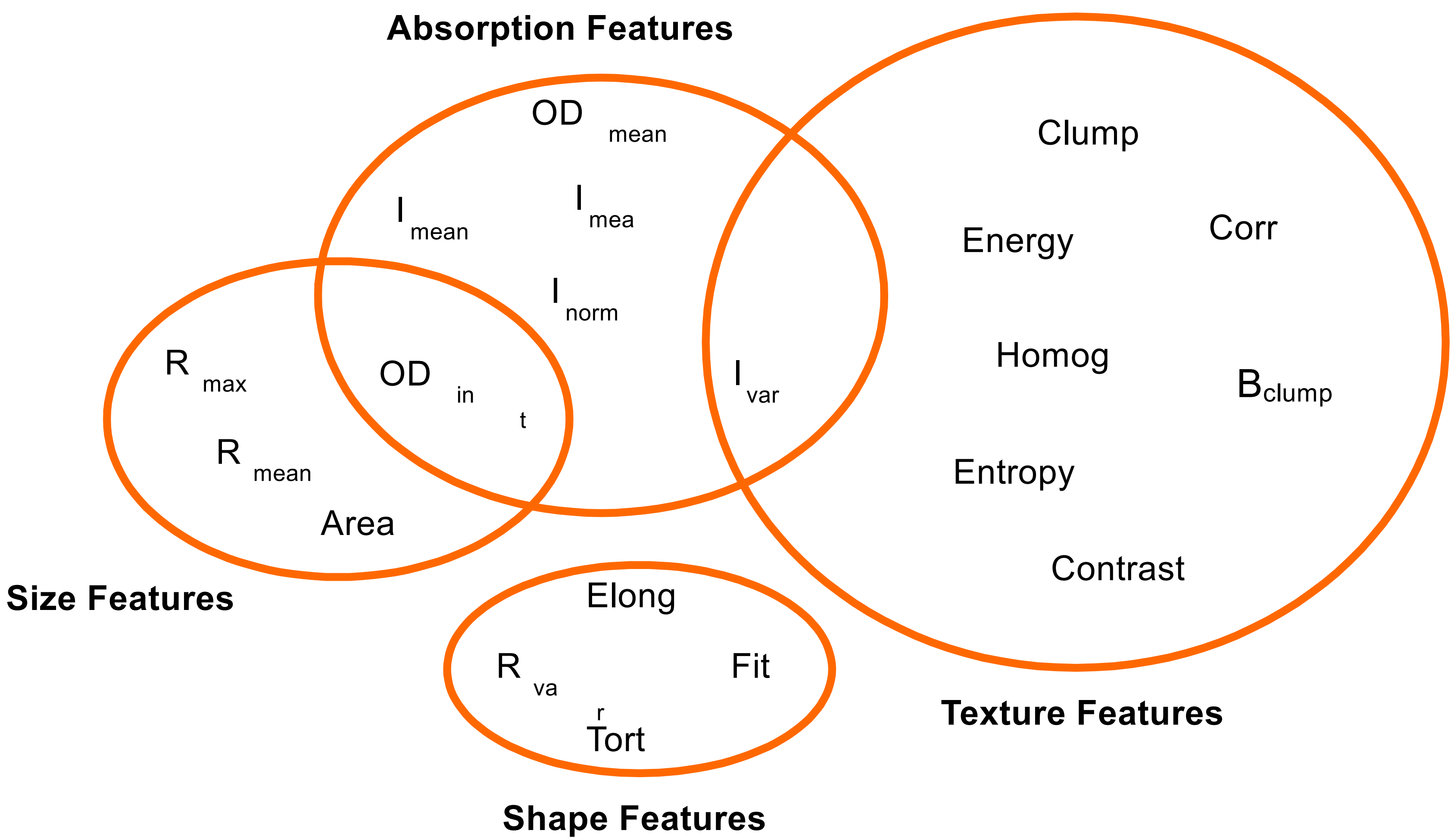
Classes:

- Carcinoma in-situ
- Light dysplastic
- Moderate dysplastic
- Normal columnar
- Normal intermediate
- <u>Normal superficial</u>
- <u>Severe dysplastic</u>



http://mde-lab.aegean.gr/downloads

Possible inputs/features



Example feature extraction

- For now we ignore segmentation mask
- Convert the gray value histogram to a feature vector
- Use the following features from the GLCM matrix:
  - Contrast
  - Dissimilarity
  - Homogeneity
  - ASM
  - Energy
  - Correlation

```python
def extract_features(image_input):
    gray =  rgb2gray(image_input)
    [counts, bins] = exposure.histogram(gray,nbins=16,source_range='dtype',
    v1 = counts.flatten()

    gray_uint = (255*gray).astype(np.uint8)
    glcm = greycomatrix(gray_uint, distances=[5], angles=[0], levels=256,
                        symmetric=True, normed=True)


    v2 = np.zeros(6)
    v2[0] = greycoprops(glcm, 'contrast')[0, 0]
    v2[1] = greycoprops(glcm, 'dissimilarity')[0, 0]
    v2[2] = greycoprops(glcm, 'homogeneity')[0, 0]
    v2[3] = greycoprops(glcm, 'ASM')[0, 0]
    v2[4] = greycoprops(glcm, 'energy')[0, 0]
    v2[5] = greycoprops(glcm, 'correlation')[0, 0]

    v = np.concatenate([v1,v2])

    return v
```

Basic ML concepts: parametric vs. non-parametric models

Parametric

Fixed number of parameters

Stronger assumptions about the nature of the data

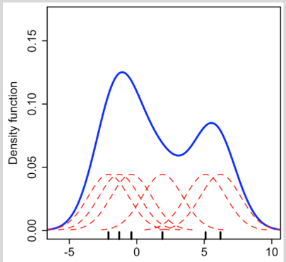
Example: k-Means

Non-Parametric

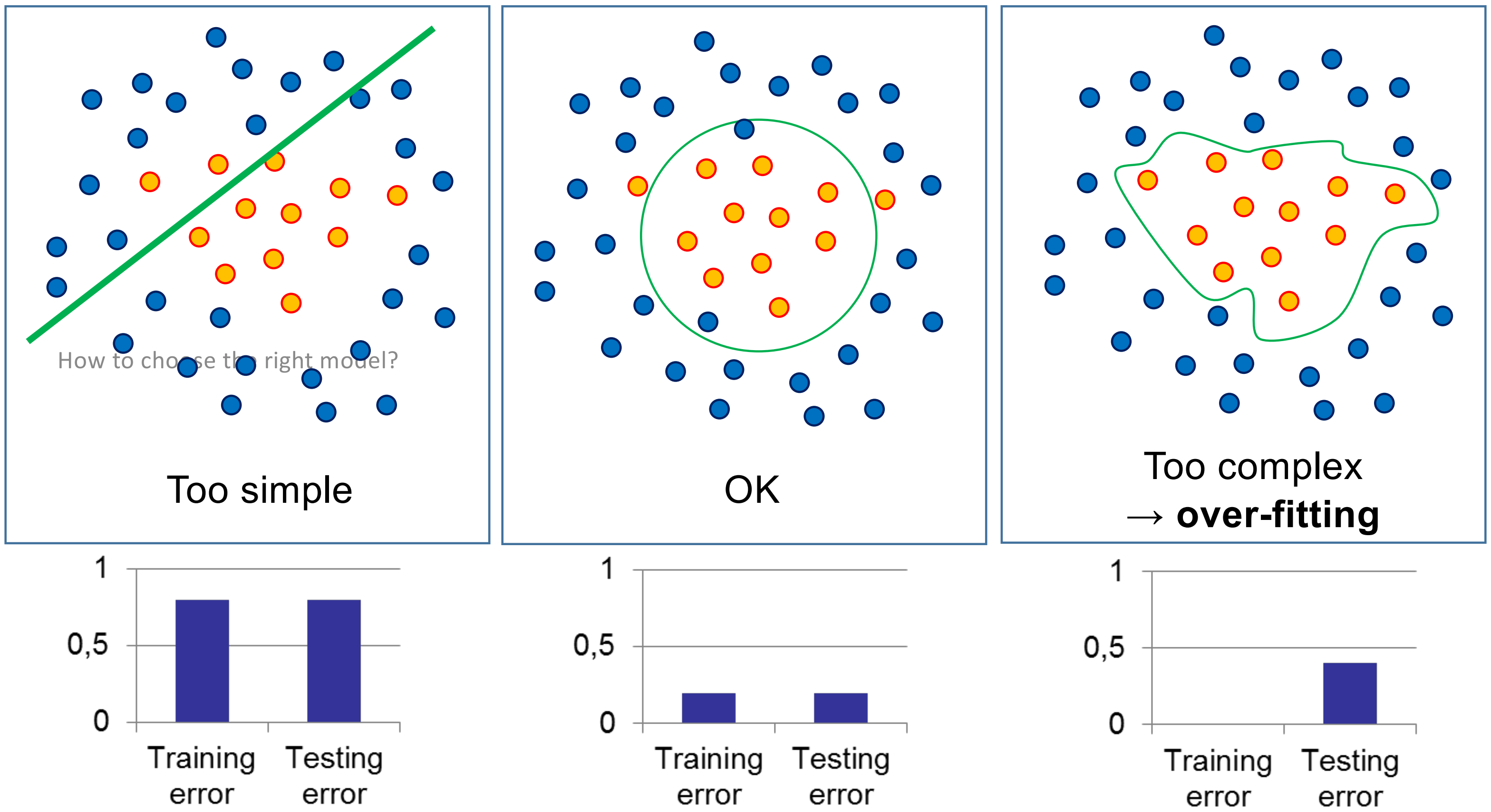
The data distribution cannot be described with a finite set of parameters

The mount of information the mode captures grows with the size of the training set.

Example: Kernel density estimators:



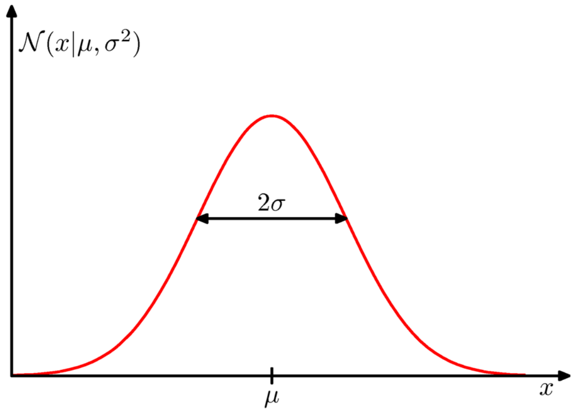
How can we select the right model?



Representing Probability Distributions

Normal or Gaussian distribution

$$\mathcal{N}(x \mid \mu, \sigma^2) = \frac{1}{(2\pi\sigma^2)^{1/2}} \exp\left(-\frac{1}{2\sigma^2}(x-\mu)^2\right)$$



- One of the most important distributions for continuous variables
- It is governed by two parameters: mean and variance

Some important properties

$$\int_{-\infty}^{\infty} \mathcal{N}(x \mid \mu, \sigma^2)\, dx = 1$$

$$\mathbf{E}(x) = \int_{-\infty}^{\infty} \mathcal{N}(x \mid \mu, \sigma^2) x\, dx = \mu$$

$$\mathbf{E}(x^2) = \int_{-\infty}^{\infty} \mathcal{N}(x \mid \mu, \sigma^2) x^2\, dx = \mu^2 + \sigma^2$$

$$\mathrm{var}(x) = \mathbf{E}(x^2) - \mathbf{E}(x)^2$$

Estimating the model parameters

- Given a set of data points that are independent and identically distributed we have

$$p(x_1, \cdots, x_K) = \prod_{i=1}^{K} \mathcal{N}(x_i \mid \mu, \sigma^2)$$

- The log likelihood of the model parameters can then be written as

$$\ln p(x_1, \cdots, x_K) = -\frac{1}{2\sigma^2} \sum_{i=1}^{K} (x_n - \mu)^2 - \frac{N}{2} \ln \sigma^2 - \frac{N}{2} \ln(2\pi)$$

Maximum likelihood estimates

Maximizing the log likelihood with respect to the mean and the variance we get

$$\mu_{ML} = \frac{1}{K}\sum_{i=1}^{K} x_i \qquad \sigma^2_{ML} = \frac{1}{K}\sum_{i=1}^{K}(x_i - \mu_{ML})^2$$

Remember: The Otsu method

Assumption: the intensity distribution consists of two classes C_0 and C_1.

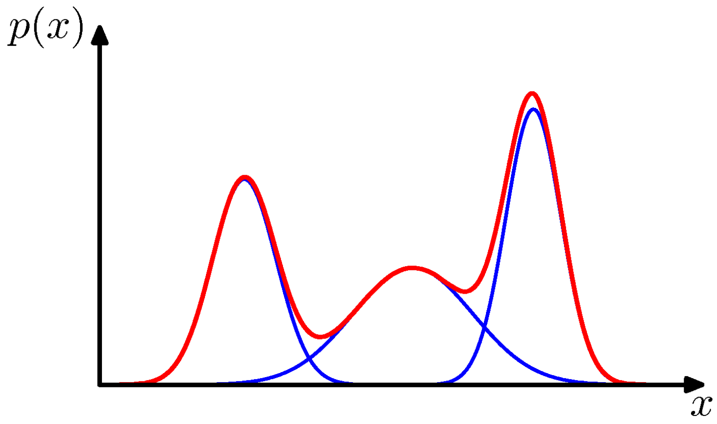
If we separate the two classes at the intensity level t we can write the class specific mean as:

$$\mu_0(t) = \sum_{i=1}^{t} iP(i|C_0) = \frac{1}{w_0(t)} \sum_{i=1}^{t} ip_i \quad \text{with} \quad w_0(t) = \sum_{i=1}^{t} p_i$$

$$\mu_1(t) = \sum_{i=t+1}^{L} iP(i|C_1) = \frac{1}{w_1(t)} \sum_{i=t+1}^{L} ip_i \quad \text{with} \quad w_1(t) = \sum_{i=t+1}^{L} p_i$$

Mixture of Gaussians

$$p(x) = \sum_{c=1}^{C} \pi_c \mathcal{N}(x \mid \mu_c, \sigma_c^2)$$



With

$$\sum_{c=1}^{C} \pi_c = 1$$

Expectation-Maximization for Gaussian Mixtures

- Initialize means and variances of the components and mixing coefficients

- Now iterate:
  - E-step: Evaluate the responsibilities for the current parameter values

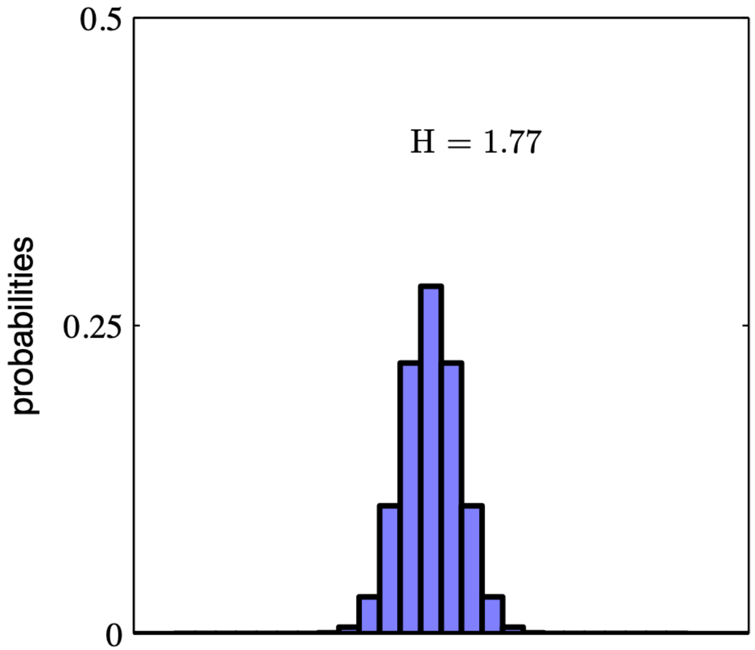
  - M-step re-estimate the parameter values

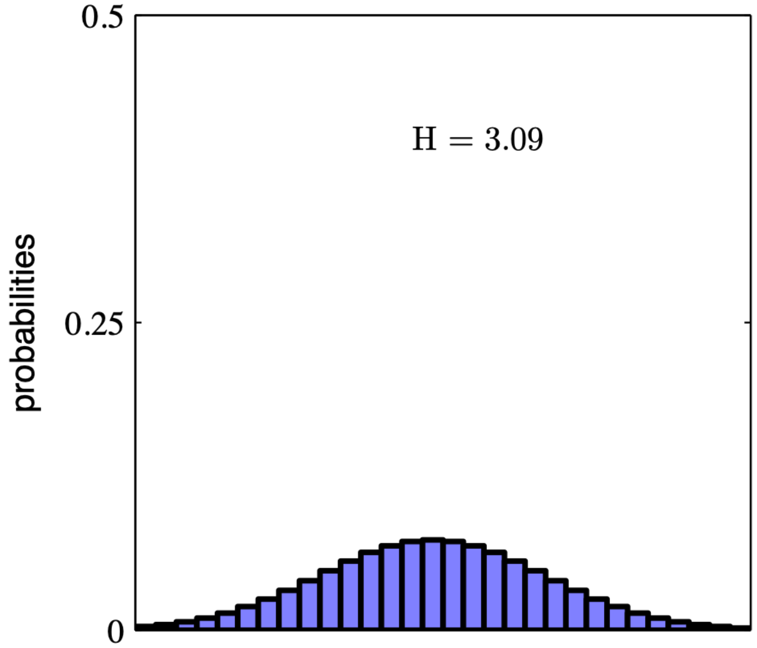
Entropy

$$H(x) = -\sum_{x} p(x) \log_2 p(x)$$

- Describes amount of information in image distribution. – The information content of an image is
  - maximal (in the information theoretic sense) if all intensities have equal probability.
  - minimal (in the information theoretic sense) if one intensity a has a probability of one, i.e. p(a) = 1.

Entropy





Kullback Leibler Divergence

$$D_{KL}(P||Q) = \int p(x) \log \left(\frac{p(x)}{q(x)} \right) \, dx$$

- The Kullback-Leibler divergence (or relative entropy) measures the difference between two probability distributions

- The definition is given for two continuous random variables. The same distance can be defined for two discrete probability distributions.

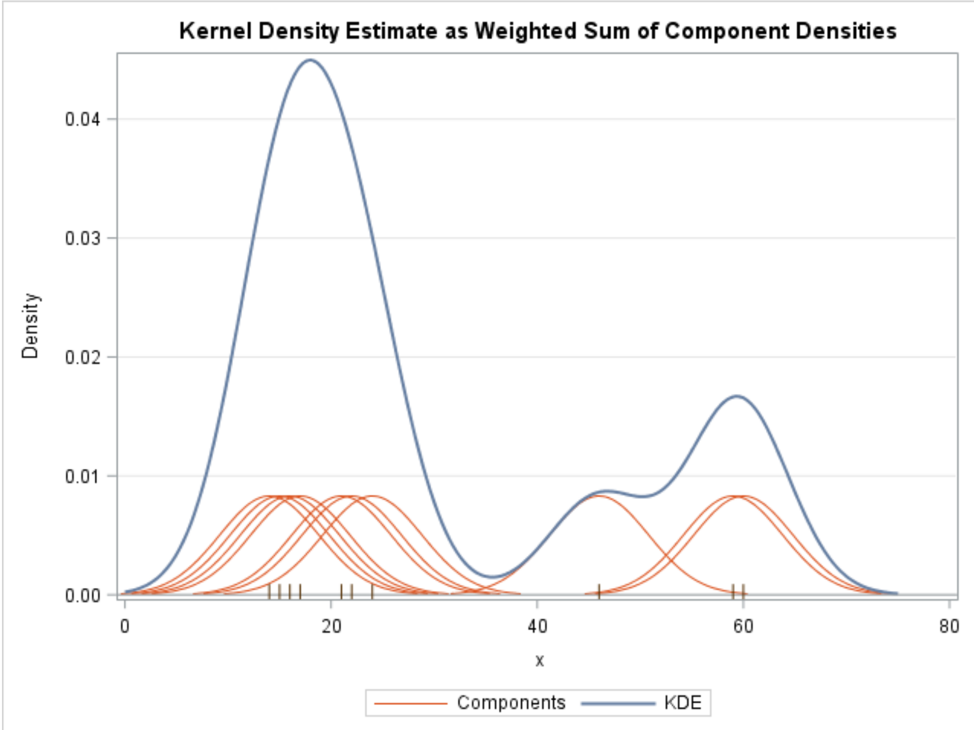
Remember: Kernel density estimation

Non-parametric way of representing a density function

Given a set of data points x_i

$$\hat{f}(x) = \frac{1}{N}\sum_{i}^{N} K_h(x - x_i)$$

Here h is the bandwidth of the kernel K



Clustering

Recap: K-means clustering

If we allocation of points to clusters would be known it would be easy to calculate the best center for each cluster.

In practice we find the best allocation of points to clusters by interating the following two steps:

- **Find the new assignment** -Assume that the cluster centers are known and allocate each point to the closest cluster
- **Find cluster centers -** Assume that the allocation is known, determine the new center for each cluster

Recap: K-means clustering

Step 1 - Initialization:

Randomly select k data points as cluster centers

Determine $\mathbf{S}^0, \mu_i^t$

Step 2 - Assignment Step:

Assign each observation to the closest cluster mean

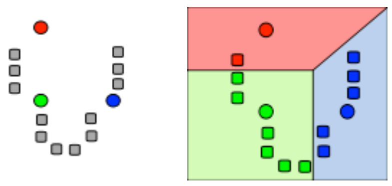
$$\min_i ||x - \mu_i^t||$$

Determine $\mathbf{S}^t$

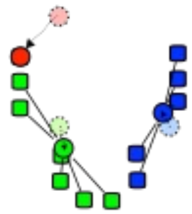
Step 3 - Update Step

Update the cluster means μ_i^{t+1}

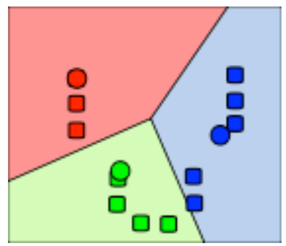
Interate Step 2 and Step 3 until convergence



Initialization



Assignment

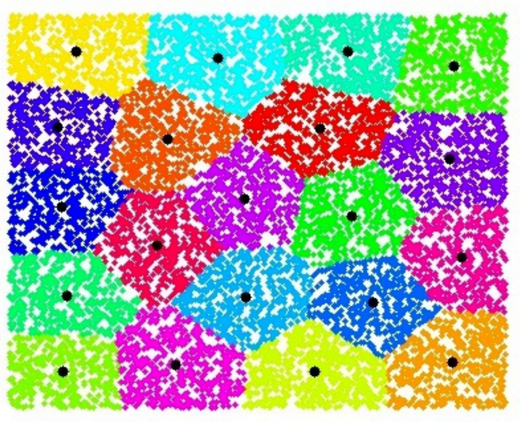


Update

K-means vs Gaussian Mixtures – the models

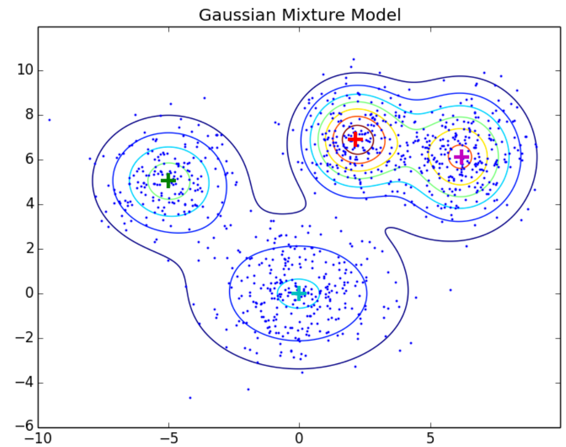
K-means

$$\sum_{\mathcal{C}_i \in \mathcal{C}} \sum_{j \in \mathcal{C}_i} ||x_j - c_i||_2$$



Gaussian Mixture Model

$$p(x|\theta) = \sum_{k=1}^{K} \pi_k \mathcal{N}(x_i|\mu_k, \Sigma_k)$$



K-means vs Gaussian Mixtures – parameter estimation

K-means

Iterating the following two steps:

Find the new assignment -Assume that the cluster centers are known and allocate each point to the closest cluster

Find cluster centers - Assume that the allocation is known, determine the new center for each cluster

Gaussian Mixture Model

Iterating the following two steps:

E-Step - computer the new weighted average of all the points assigned to a given cluster.

M-Step - find the new parameters for each of the components

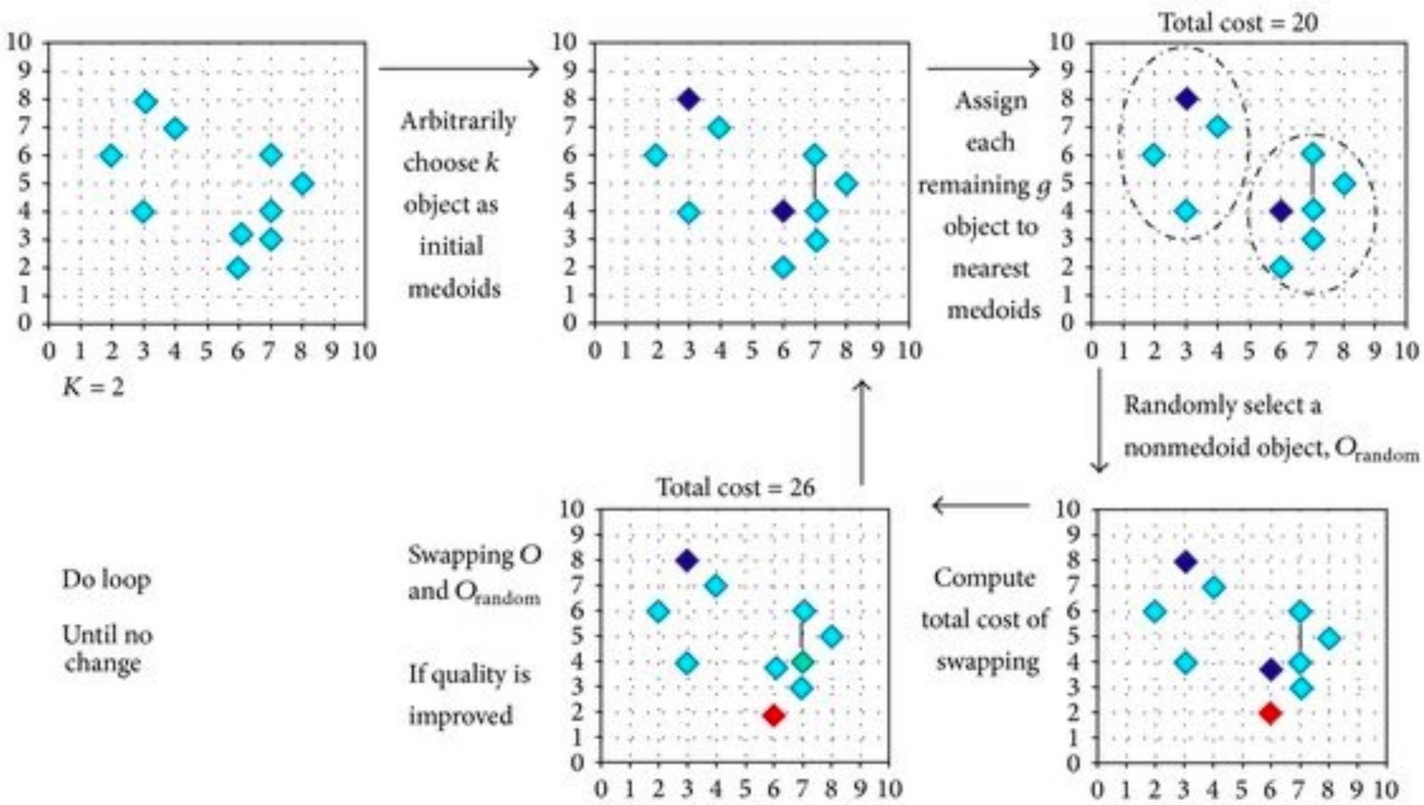
K-medoids

- In k-means we are calculating the mean, that means that the cluster is represented by an interpolated instance

- Instead of representing each cluster's centroid by the mean of all the data assigned to this cluster, the centroid will be one of the data points.

$$m_k = \underset{i \in \mathcal{K}}{\operatorname{argmin}} \sum_{i \in \mathcal{K}} d(i, j)$$

K-medoids


K = 2
Arbitrarily choose k object as initial medoids
Assign each remaining g object to nearest medoids
Total cost = 20
Randomly select a nonmedoid object, O_{random}
Compute total cost of swapping
Total cost = 26
Swapping O and O_{random}
If quality is improved
Do loop
Until no change

Dimensionality Reduction

Dimensionality Reduction

Projection methods are a vital for visualising high dimensional data.

One ore more linear combinations of the original features are chosen to maximize some measure of “interestingness”.

The most popular methods are:

- Principal components
- Multi-dimensional scaling

Sample Covariance

The covariance matrix for a random variable

$$x \in \mathcal{R}^n$$

is a square symmetric matrix

$$C \in \mathcal{R}^{n \times n} \qquad \text{with} \qquad C_{i,j} = \sigma(x_i, x_k)$$

It can be computed as

$$C = \frac{1}{n-1} \sum_{i=1}^{n} (X_i - \bar{X})(X_i - \bar{X})^T$$

Principal component analysis

Goal: Find a linear projection that approximates the data in a lower dimensional space

The singular value decomposition of the covariance matrix of X with mean zero is one way to express the principle components of the data.

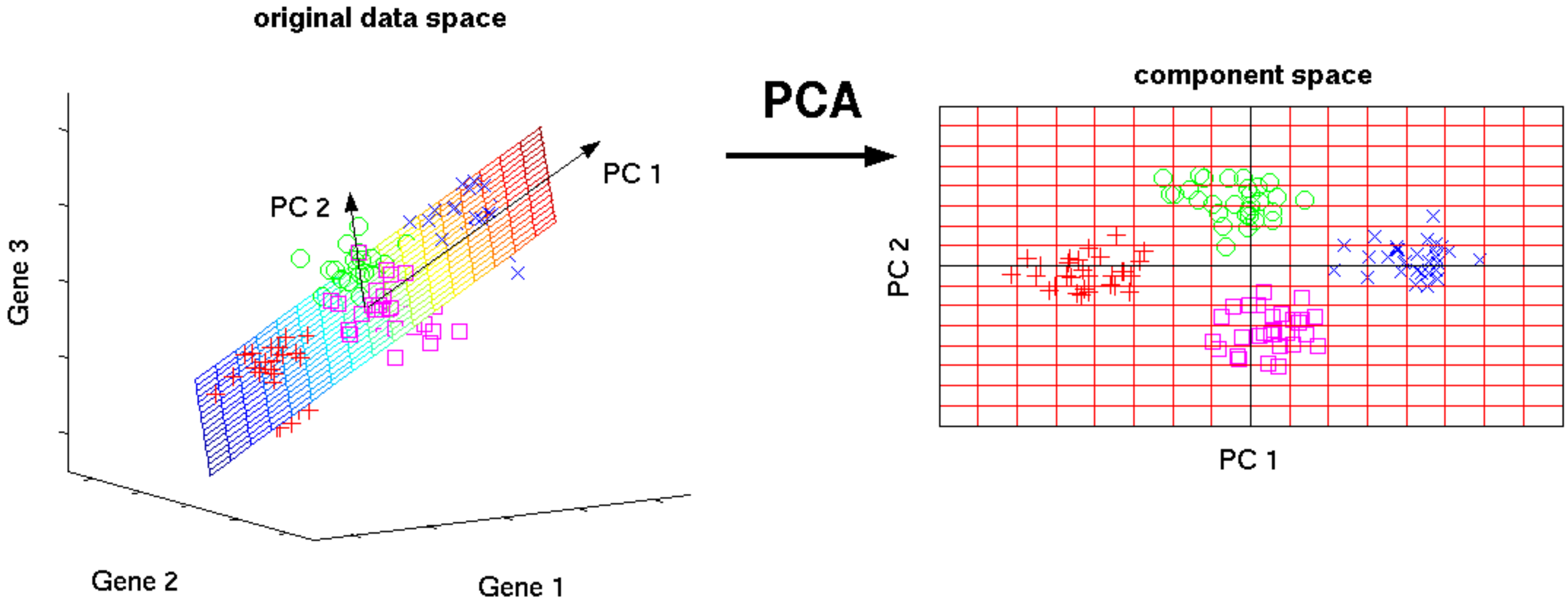
$$X^T X = V \Lambda V^T \quad \text{where } \Lambda \text{ is a diagonal matrix}$$

The eigenvectors v of V are called principal components.

Principal component analysis

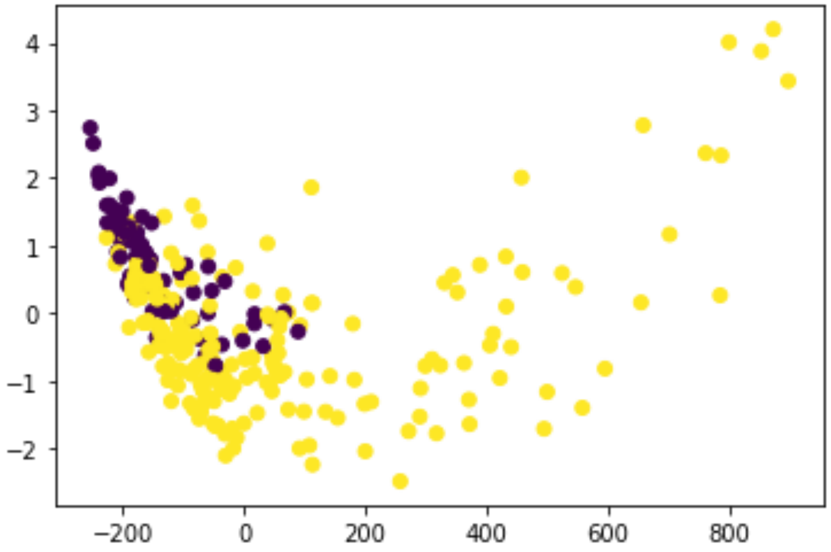
- The first singular value (the first column of V) is the linear combination $a^T x$ for a of unit length with the largest variance
- The first $q<p$ columns of V are the linear projection of X into q dimensions with the largest variance.
- If we project onto the first q principal components we have the most accurate rank-q reconstruction of the original data.
- Unless we have prior information we scale the variance of the features to be comparable.

Principal component analysis



Example: PCA on pap smear data

First and second component of the PCA on the pap smear data



Multidimensional Scaling

We would like to find values z to minimise

$$S_M(z_1, \cdots, z_N) = \sum_{i \neq j} (d_{ij} - ||z_i - z_j||)^2$$

This method is known as least squares scaling.

A gradient decent algorithm is used to find

Many other variants of this approach have been explored.

Multidimensional scaling

Goal: Visualise similarities in a data set. Every object should be placed in an n-dimensional data set such that the between-object object distances are preserved as well as possible.

Given a set of dissimilarities:

$$\mathbf{D} = \{d_{ij}\}$$

We would like find

$$z_1, \cdots, z_N \in \mathcal{R}^k$$

that represent the original data point in a space of dimension k < n.